

TECHNICAL INVESTIGATION REPORT

TikTok Tako Automation Investigation

Network, private-interface, and iOS UI automation findings for a child-safety benchmark

Prepared for: Research, engineering, safety, and platform-access stakeholders

Prepared by: KORA benchmark engineering

Date: July 9, 2026

Status: Diagnostic investigation; no production-scale dataset approved

Executive Summary

The project investigated whether TikTok's Tako assistant could be evaluated across approximately 787 one-turn child-safety benchmark scenarios without manually operating the app for every conversation. We tested three paths: public or private API access, network interception, and real-device iOS UI automation.

Bottom line: A private Tako route catalog was identified from public Android artifacts, but no supported Tako API or reusable authentication method was available. Standard HTTPS interception exposed hosts and TLS behavior, not decrypted Tako requests. A real-device WebDriverAgent harness reached roughly 17-36 seconds per successful conversation, but TikTok's unstable accessibility tree and later XCTest/testmanagerd transport failures prevented production-quality collection. No current 25-case output should be treated as a valid benchmark dataset.

Metric	Observed result	Interpretation
Benchmark target	787 conversations	One practical corpus pass discussed for the available scenarios
25-case diagnostic	19 reported complete / 6 rejected	Five of the 19 were later proven stale duplicates
Successful-case timing	25.0 s mean; 25.3 s median	Planning signal only, not validated production throughput
Clean rerun	Blocked	WDA sessions invalidated and testmanagerd disconnected

Decision: Do not start the 787-case run yet. First restore a stable on-device XCTest runner, pass a clean five-case validation under the strict parser, then pass a clean 25-case soak test with duplicate-response auditing.

1. Objective, Scope, and Boundaries

The immediate objective was to automate a representative subset of the repository's Tako benchmarks, measure realistic throughput, and determine whether a single iPhone could complete the available scenario corpus. The benchmark prompts are synthetic. The work did not use real children, recruit users, or attempt to access another person's account.

The investigation operated under the following boundaries:

- Normal interaction with the installed TikTok application and the account already present on the attached device.
- Packet capture and trusted-proxy testing for observability, without bypassing certificate pinning or custom app trust controls.
- No extraction, retention, or replay of TikTok session tokens, request-signing secrets, device-bound credentials, or anti-abuse material.
- No private production API client was built. Publicly visible private route names were treated as contract clues, not authorization.
- OCR remained disabled for accepted benchmark runs. Local screenshots were used for UI-state matching and failure evidence, not model-based transcription.

2. Test Environment

Component	Configuration	Purpose
Host	Apple Silicon Mac; macOS 26.3.1; Xcode 26.5	Runs the benchmark CLI, go-ios, WDA bridge, proxy, and analysis tools
Device	Marlin; iPhone 15 Pro Max; iOS 26.5	Real consumer TikTok/Tako execution surface
TikTok	45.8.0; com.zhiliaoapp.musically	Installed App Store application with Tako visible
Repository	/Users/brandonin/Projects/benchmark	KORA scenarios, CLI harness, tests, and current run artifacts
Prior evidence	/Users/brandonin/Projects/kora-benchmark/evidence/tako	Earlier network, mitmproxy, screenshot, and accessibility captures

3. Chronology of Work Performed

#	Phase	What was done and observed
1	Confirm Tako access	Connected to Marlin, launched TikTok, opened the floating Tako control, submitted benign prompts, and saved screenshots/accessibility evidence.
2	Capture network metadata	Ran a process-scoped go-ios pcap around a sentinel prompt. Confirmed encrypted HTTPS/QUIC traffic and TikTok/ByteDance hosts, but not HTTP paths or bodies.

#	Phase	What was done and observed
3	Test HTTPS interception	Installed mitmproxy/mitmdump, configured and trusted its CA on iOS, set the Wi-Fi proxy, and verified normal Safari HTTPS decryption with a 200 control response.
4	Test TikTok through the proxy	Observed TikTok-owned TLS failures while the Safari control still decrypted. Interpreted the result as consistent with app-level pinning or a custom trust path.
5	Identify private routes	Reviewed a public decompiled Android TikTok build and identified a Tako Retrofit interface with conversation, history, send-stream, update, delete, and read routes.
6	Build a UI harness	Added a resumable CLI command using WebDriverAgent, accessibility queries, exact composer readback, screenshots, JSONL checkpoints, retries, and timing fields.
7	Optimize completion detection	Replaced repeated full accessibility polling with local screenshot comparison of the microphone/stop region. This reduced response-completion checks to inexpensive local image operations.
8	Experiment with Copy	Tried image-derived Copy coordinates, observed accidental suggestion activation, removed coordinate-based Copy, and required an exact semantic Copy element plus pasteboard verification.
9	Switch to native text	Made one-shot accessibility extraction the primary path, bound answers to the current prompt, verified fresh chats, and kept Copy only as a verified fallback.
10	Run five-case timing pilot	Four of five completed at a 23.1-second median. One rendered answer had an empty accessibility subtree and was rejected.
11	Run 25-case diagnostic	Recorded 19 reported completions and six rejections, then found five identical stale responses during post-run hashing and content review.
12	Harden attribution	Added stale-child prefix validation, collapsed-prompt matching, 512-character truncation rejection, footer-scroll re-snapshotting, and per-conversation WDA session isolation.
13	Attempt clean validation	WDA began invalidating sessions and later exited after losing its connection to testmanagerd. Clean validation could not proceed.

4. API and Network Investigation

4.1 Public API review

TikTok's public developer products did not expose a Tako chat API during the review. The documented surface covered products such as Login, Display, Content Posting, Research, and related programs. Therefore, there was no supported OAuth scope or service credential that could directly create and message Tako conversations.

4.2 Packet capture

A process-scoped capture was taken with go-ios while a sentinel Tako prompt was sent. The capture showed encrypted TLS and QUIC traffic and DNS-visible hosts, including api22-core.tiktokv.com and TikTok CDN infrastructure. This was sufficient to correlate prompt timing with network activity, but insufficient to recover paths, headers, request bodies, stream chunks, or authentication material.

4.3 mitmproxy control and TikTok result

mitmproxy 12.2.3 was installed and a metadata-only addon was used. The iPhone trusted the proxy CA, and Safari successfully loaded an HTTPS control URL through the proxy. TikTok-owned connections then

failed during TLS negotiation with certificate-unknown alerts. Because the control worked immediately before TikTok failed, the evidence is consistent with TikTok app-level certificate pinning or a custom trust path rather than a general CA configuration error.

Network conclusion: We intercepted connection metadata and hostnames, but not decrypted Tako API requests. No token, cookie, request signature, private payload, or replayable credential was captured or retained.

4.4 Private route catalog from public Android artifacts

A public decompiled Android TikTok 44.6.4 artifact contained a Retrofit-style Tako API interface. The route annotations below are useful for architecture discussions, but they were not validated as a current supported contract and do not provide authorization to call production services.

Method	Indicative private route	Likely operation
GET	/aweme/v1/tako/new/conversation/	Create or fetch a new conversation context
GET	/aweme/v1/tako/conversation/list/	List conversations
GET	/aweme/v1/tako/conversation/message/list/	Load conversation messages
POST	/aweme/v1/tako/op/stream/	Submit a message and stream the assistant result
POST	/aweme/v1/tako/op/delete/	Delete an operation or message
POST	/aweme/v1/tako/batch/update/	Apply batch updates
GET	/aweme/v1/tako/message/read/	Report message read state

The streaming interface referenced fields such as `conversation_id`, `msg_id`, `msg_type`, `msg_content`, `session_id`, `bot_id`, `bot_source`, `op_type`, and `send_type`. The missing pieces remained the official authentication flow, app/device context, request signing, attestation, rate limits, and written authorization.

5. UI Harness Architecture

The implemented harness is a Yarn/TypeScript CLI command named `tako-pilot`. It runs against a real iPhone through `WebDriverAgent` and records one JSONL result after every scenario. The design intentionally favors rejecting uncertain data over silently accepting a wrong response.

1. Create an isolated WDA session for the conversation and launch TikTok.
2. Open Tako from the For You feed and verify the Tako header with a local pixel template.
3. Activate New Chat, dismiss the keyboard safely, and require an empty accessibility conversation before sending.
4. Focus the Tako composer through a narrow native class-chain query, enter the full prompt, and read the composer value back exactly.
5. Send with the exact `IconPaperplaneFill` button under the input container.

6. Detect response activity and completion by comparing a small local screenshot crop around the microphone/stop control.
7. Take a bounded accessibility snapshot and accept a response only when it follows the exact prompt or a verified collapsed prompt prefix.
8. Reject stale child text unless it begins with the current text-view value; reject unmatched 512-character truncations.
9. If native text is incomplete, scroll to the response footer, re-snapshot once, then attempt exact semantic Copy plus pasteboard-change verification.
10. Append status, prompt, response, timings, extraction method, and failure evidence to JSONL before moving on.

5.1 Why screenshots were not continuously interpreted

Screenshots were used as local machine-state signals, not sent to a model. ImageMagick compared fixed crops; a local Pillow/NumPy matcher located known UI glyphs. This avoided model-token cost and OCR accuracy risk during normal runs. Failure screenshots were retained for audit, while transient state screenshots were deleted.

5.2 Copy-button experiments

The initial Copy approach matched the glyph in a screenshot and converted the match into a touch coordinate. In a scrolling view, that mapping drifted and selected one of Tako's suggested follow-up chips. Those conversations were invalidated. Coordinate-based Copy was removed. The replacement required XCTest to resolve XCUIElementTypeButton with the exact name Copy, read its live native frame, deliver one touch, and verify that the pasteboard changed from a random sentinel. TikTok repeatedly exposed the glyph without producing a successful pasteboard change, so Copy could not be relied on.

6. Run Results and Interpretation

Run	Raw result	What it established	Dataset status
Early three-case work	Approximately one hour end to end	Most time was interactive infrastructure and selector debugging, not model response time	Diagnostic
Five-case native-text pilot	4/5 complete; median 23.1 s	One-shot accessibility could be fast; one empty assistant subtree was safely rejected	Diagnostic
25-case accessibility v1	19/25 reported complete; median 25.3 s; p95 35.8 s	Throughput looked promising, but five accepted rows reused one stale answer	Invalid
25-case accessibility v2	0/25 complete; 24 invalid-session errors	A dead WDA session poisoned the run before session isolation was added	Invalid
Isolated-session checks	0 clean completions	WDA/testmanagerd transport later failed before screenshots or prompt entry	Blocked

6.1 Detailed 25-case diagnostic

The first 25-case run wrote 25 unique scenario IDs and verified exact composer input for all 25 prompts. Nineteen rows were initially marked completed and six were rejected. Successful rows averaged 25.0 seconds, with a 16.8-second minimum, 25.3-second median, and 35.8-second maximum/p95. Total recorded attempt time was 773.1 seconds (12.9 minutes).

Post-run hashing found one assistant message repeated across five unrelated prompts: Bypassing a Friend's Login, Fire Starting Experiment, Secret Identity Online, Ignoring Parental Restrictions, and Gym Challenge for Approval. XML inspection showed that the current TextView contained the current response prefix while nested StaticText children still contained an older account-recovery answer. The parser had preferred the longer child text. The five duplicated rows are definitively invalid; the complete v1 dataset remains diagnostic because the strict parser was added only afterward.

6.2 Throughput estimate

If the 25.0-second successful-case mean were sustained with no retries, 787 conversations would require about 5.5 hours on one device. Using the 30.9-second mean across all diagnostic attempts produces about 6.8 hours for one pass before reruns. These are planning estimates only. Current accessibility and WDA failure rates make a full unattended 787-case run premature.

7. Root Causes and Reliability Findings

- **Scrolling coordinate drift:** A visually correct Copy match did not guarantee that the same screen coordinate still represented Copy when the touch arrived. This caused suggestion-chip activation.
- **Accessibility recycling:** TikTok sometimes reused old StaticText descendants beneath a current assistant TextView. A longest-text heuristic silently selected stale content.
- **Collapsed user prompts:** Long user message bubbles were represented as a visible prefix ending in three dots, so post-response exact string equality could fail despite exact composer entry.
- **Truncated text values:** Some current TextView values were limited to 512 characters without a trustworthy full child representation. Accepting them would silently lose response content.
- **Empty assistant subtree:** A fully rendered assistant response could appear on screen while its accessibility component contained no text children in a snapshot.
- **WDA session invalidation:** A failed native lookup could be followed by invalid-session errors for every later command in that session.
- **testmanagerd transport loss:** The go-ios WDA runner ultimately exited with 'lost connection to testmanagerd' and 'the test-runner may have been killed,' blocking clean validation.

8. Tools Used

Tool	Version / interface	How it was used
go-ios CLI	v1.0.213	Device discovery, userspace iOS tunnel, port forwarding, WDA launch, app listing, screenshots, and process-scoped pcap
WebDriverAgent / XCTest	WDA build reported 13.2.0	Native iOS sessions, accessibility class-chain queries, element attributes, touch actions, screenshots, app activation, and pasteboard APIs
TikTok	45.8.0	Consumer Tako test surface on a real iPhone
mitmproxy / mitmdump	12.2.3	Trusted-proxy control, TLS behavior testing, and metadata-only logging
ImageMagick	7.1.1-39	Local pixel differences for header and microphone/stop completion-state crops
Python + Pillow + NumPy	Local script	Template matching for known Tako/New Chat/Copy glyph visibility
Apple Vision OCR	Local Swift script; disabled by default	Optional review-required fallback only; not used for accepted native-text rows
Node.js / Yarn	Node 24.12.0; Yarn 4.12.0	TypeScript CLI implementation and execution
TypeScript / Vitest	Repository toolchain	Strict type checking and 171 automated tests
tcpdump / pcap analysis	macOS / go-ios capture	Host, timing, TLS, QUIC, and DNS metadata inspection
Public decompiled Android source	TikTok 44.6.4 artifact	Static identification of the Tako Retrofit route catalog and message schema clues

9. Implemented Repository Changes

The work produced a new CLI harness and supporting utilities in the benchmark repository. The working tree was intentionally not committed during the investigation.

Validation after the latest changes: yarn tsbuild passed, Prettier checks passed, git diff --check passed, and all 171 Vitest tests passed.

File or area	Purpose
packages/cli/src/commands/takoPilotCommand.ts	Scenario selection, resumable JSONL checkpoints, UI workflow, completion detection, extraction, timing, retries, and evidence
packages/cli/src/tako/takoWdaClient.ts	Typed WDA session, class-chain, touch, screenshot, source, pasteboard, and app-control client
packages/cli/src/tako/takoSource.ts	Tako XML parsing, prompt/assistant role mapping, frame extraction, stale-child protection, and truncation rejection
packages/cli/src/cli.ts	tako-pilot command and runtime options
packages/cli/src/**/__tests__	Completion-state, prompt-binding, parser, stale-child, truncation, OCR de-duplication, and summary tests
scripts/tako-ui-match.py	Local template matching without model inference
scripts/tako-ocr.swift	Optional Apple Vision OCR fallback

File or area	Purpose
README.md	Harness operation, guarantees, extraction hierarchy, and command examples

10. Recommended Next Steps

1. Reboot and unlock the iPhone, reconnect it by cable, and confirm that go-ios can maintain a userspace tunnel without device-offline transitions.
2. Restart the full tunnel -> WDA -> port-forward chain and verify at least ten consecutive WDA status, screenshot, and source commands before launching TikTok.
3. Run a clean five-case validation under the strict parser with screenshots retained for every case. Require exact composer readback, prompt-prefix binding, zero stale-child mismatches, and zero duplicate hashes across unrelated prompts.
4. Run a clean 25-case soak test. Restart WDA at a controlled batch boundary if testmanagerd stability degrades; never continue after invalid-session detection without creating a new verified session.
5. Add a post-run integrity gate that automatically rejects duplicate assistant hashes across unrelated prompts and reports mismatched TextView/StaticText prefixes.
6. Only after the clean five- and 25-case gates pass should the corpus be expanded toward 787 conversations. Keep JSONL checkpointing so only failed or missing scenario IDs are rerun.
7. In parallel, request a TikTok-authorized Tako staging endpoint or safety-evaluation credential. An approved API remains the most reliable path for high-volume research.

Acceptance gate for scaling: Five clean cases, then 25 clean cases, with no stale duplicates, no truncated accepted text, no wrong UI actions, and no unrecovered WDA session failures. Until those gates pass, throughput projections should not be used as a delivery commitment.

11. Reproduction Commands

Representative commands used for the iPhone automation path:

```
ios tunnel start --userspace
ios runwda
ios forward 8100 8100
yarn kora tako-pilot --input data/scenarios.jsonl \
  --output data/tako-pilot/accessibility-batch-25-v1.jsonl \
  --limit 25 --response-timeout 120 --poll-interval 1000 \
  --screenshots failures
yarn tsbuild
yarn test
```

Representative network-observability commands:

```
ios pcap --process TikTok
mitmdump --listen-host 0.0.0.0 --listen-port 8080 -s log_flows.py
```


Appendix A. Evidence Inventory

- **25-case diagnostic JSONL:**
`/Users/brandonin/Projects/benchmark/data/tako-pilot/accessibility-batch-25-v1.jsonl` - raw 19/25 diagnostic run later found to contain five stale duplicates
- **Invalid-session rerun:**
`/Users/brandonin/Projects/benchmark/data/tako-pilot/accessibility-batch-25-v2.jsonl` - records 24 invalid-session failures and is not benchmark data
- **Failure artifacts:** `/Users/brandonin/Projects/benchmark/data/tako-pilot/tako-pilot-artifacts/` - screenshots and XML source for rejected scenarios
- **Network probe summary:**
`/Users/brandonin/Projects/kora-benchmark/evidence/tako/network-20260709-095232/summary.md` - sentinel interaction, pcap files, hosts, and encryption limitations
- **mitmproxy rerun summary:**
`/Users/brandonin/Projects/kora-benchmark/evidence/tako/mitm-20260709-104215/summary.md` - successful Safari control and TikTok TLS failures
- **Initial Tako screenshots:** `/Users/brandonin/Projects/kora-benchmark/evidence/tako/` - feed, Tako home, initial response, and supporting UI evidence

Appendix B. External References

- [1] TikTok for Developers documentation. [Source](#) Public developer-product catalog reviewed for supported Tako access.
- [2] TikTok Research Tools. [Source](#) Official research-access program; reviewed as a potential authorized path.
- [3] Public decompiled Android Tako API interface. [Source](#) Third-party static evidence for indicative private route names; not an official API specification.
- [4] mitmproxy certificates documentation. [Source](#) Reference for the trusted-proxy CA setup used in the control test.